

			成 绩	
			评阅人	

复 旦 大 学

本 科 生 课 程 实 验 报 告

论文题目:	基于残差卷积神经网络的组织切片病理图像分类研究
修读课程:	人工智能的编程基础
选课学期:	2025-2026 学年第二学期
选课学生:	杨博凯, 李林蔚, 陈睿尹
完成日期:	2026 年 6 月 12 日

目录

1	引言	3
2	数据集与任务	3
2.1	数据概况	3
2.2	数据划分与增强	4
3	模型架构设计	6
3.1	整体结构	6
3.2	残差双卷积块 (ResConv Block)	7
3.3	残差连接的核心作用	8
3.4	全局平均池化 (Global Average Pooling)	9
3.5	分类器	9
4	类别不平衡处理策略	9
4.1	问题分析	9
4.2	本模型的策略：仅用 pos_weight	9
4.3	为什么不用 WeightedRandomSampler?	10
5	训练策略	11
5.1	学习率调度	12
5.2	训练循环主流程	13
5.3	模型评估与最优阈值搜索	14
6	实验结果	16
6.1	最优模型测试集性能	16
6.2	超参数控制变量实验	17
6.2.1	学习率的影响	17
6.2.2	Dropout 的影响	18
6.2.3	Batch Size 的影响	19
6.3	训练曲线分析	21
6.3.1	训练与验证 Loss 曲线	21
6.3.2	训练与验证 Accuracy 曲线	22
6.4	各实验结果汇总	22
7	讨论	23
7.1	架构设计的合理性	23
7.2	性能天花板	23

目 录	2
8 结 论	23
附录 小组成员分工与协同研发说明	24

摘要

本报告针对医学组织切片图像的良/恶性二分类任务，设计并实现了一个基于残差双卷积块（ResConv Block）与全局平均池化（Global Average Pooling）的轻量级卷积神经网络，参数量约 1.25M。训练阶段采用按患者分组的 GroupShuffleSplit 数据划分、Warmup + Cosine Annealing 学习率调度、温和的 pos_weight 类别不平衡补偿策略（无 WeightedRandomSampler），并通过控制变量法对学习率、Dropout 和 Batch Size 进行了系统的超参数优化实验。最终在 Batch Size = 32 的配置下取得测试集准确率 90.30%、AUC 0.9581 的优异性能，训练过程稳定、收敛良好、无明显过拟合，验证了架构设计与训练策略的合理性。

关键词：医学图像分类；残差卷积网络；全局平均池化；类别不平衡；Batch Size 优化；Warmup 预热

1 引言

本报告解答了编程项目中的第二道题：“Breast Histopathology Images”。实验针对医学组织切片图像的良/恶性二分类任务，设计并训练了一个基于残差卷积网络的深度学习模型。最终在测试集上达到了 90.30% 的准确率和 0.9581 的 AUC 值。本报告详细阐述模型的架构设计、训练策略，以及超参数控制变量实验的结果与分析。

2 数据集与任务

2.1 数据概况

数据集包含 279 位患者的 277,524 张组织切片图像，标签为二分类：正常（class 0）与患病（class 1）。原始图像尺寸为 50×50 RGB。类别不平衡比例为 2.52:1（正常 198,738 张，患病 78,786 张），属于中等程度的不平衡。

不同患者之间的类别分布差异极大——某些患者有超过 2,000 张正常切片但仅 20 张病灶切片，而另一些患者病灶切片多于正常切片。这种患者级别的异质性是医学图像分类的典型挑战。

数据集的目录结构为 `archive/<patient_id>/<label>/*.png` (label 0 = 正常, label 1 = 患病)，读取逻辑如下：

```
dataset.py —collect_samples

1 def collect_samples(data_dir: str = DATA_DIR) -> list[dict]:
2     """遍历 archive 目录，收集所有样本的元信息。"""
3     samples = []
4     for patient_id in sorted(os.listdir(data_dir)):
```

```

5     patient_dir = os.path.join(data_dir, patient_id)
6     if not os.path.isdir(patient_dir):
7         continue
8     for label_str in ("0", "1"):
9         label_dir = os.path.join(patient_dir, label_str)
10        if not os.path.isdir(label_dir):
11            continue
12        for fname in os.listdir(label_dir):
13            if fname.lower().endswith(".png"):
14                samples.append({
15                    "path": os.path.join(label_dir, fname),
16                    "label": int(label_str),
17                    "patient_id": patient_id,
18                })
19    return samples

```

2.2 数据划分与增强

采用按患者分组的 GroupShuffleSplit 划分策略，确保同一患者的切片不会同时出现在训练集和验证/测试集中，以避免数据泄漏：

表 1: 数据集划分（按患者分组）

子集	患者数	样本数	Class 0	Class 1
训练集	167	168,318	120,528	47,790
验证集	56	55,738	39,471	16,267
测试集	56	53,468	38,739	14,729

划分采用两步 GroupShuffleSplit——先从全体样本分出 60% 训练集与 40% 临时集，再将临时集对半分为验证集与测试集：

dataset.py —split_data（核心划分逻辑）

```

1 def split_data(samples: list[dict]):
2     n = len(samples)
3     indices = np.arange(n)
4     groups = np.array([s["patient_id"] for s in samples])
5
6     # Step 1: train (60%) vs temp (40%)
7     gss1 = GroupShuffleSplit(n_splits=1, test_size=VAL_RATIO + TEST_RATIO,

```

```

8         random_state=SPLIT_SEED)
9     train_idx, temp_idx = next(gss1.split(indices, groups=groups))
10
11     # Step 2: val (20%) vs test (20%) from temp
12     temp_groups = groups[temp_idx]
13     temp_indices = indices[temp_idx]
14     gss2 = GroupShuffleSplit(n_splits=1,
15                             test_size=TEST_RATIO / (VAL_RATIO +
16 TEST_RATIO),
17                             random_state=SPLIT_SEED)
18     val_idx_in_temp, test_idx_in_temp = next(gss2.split(temp_indices,
19 groups=temp_groups))
20     val_idx = temp_indices[val_idx_in_temp]
21     test_idx = temp_indices[test_idx_in_temp]
22     # ... 返回 train_samples, val_samples, test_samples

```

训练时的数据增强包括随机水平翻转 ($p = 0.5$)、随机垂直翻转 ($p = 0.5$)、随机旋转 ($\pm 30^\circ$, $p = 0.5$) 和颜色抖动 (brightness=0.1, contrast=0.1, $p = 0.5$)。验证集和测试集仅做归一化处理，不做增强。所有图像在输入模型前被缩放至 64×64 。归一化采用 ImageNet 的均值 (0.485, 0.456, 0.406) 和标准差 (0.229, 0.224, 0.225)。

```

dataset.py —get_transforms
1 def get_transforms(train: bool = True) -> A.Compose:
2     if train:
3         return A.Compose([
4             A.Resize(IMAGE_SIZE, IMAGE_SIZE),
5             A.HorizontalFlip(p=0.5),
6             A.VerticalFlip(p=0.5),
7             A.Rotate(limit=30, p=0.5),
8             A.ColorJitter(brightness=0.1, contrast=0.1, p=0.5),
9             A.Normalize(mean=IMAGE_MEAN, std=IMAGE_STD),
10            ToTensorV2(),
11        ])
12     else:
13         return A.Compose([
14             A.Resize(IMAGE_SIZE, IMAGE_SIZE),
15             A.Normalize(mean=IMAGE_MEAN, std=IMAGE_STD),
16            ToTensorV2(),

```

17

1)

3 模型架构设计

3.1 整体结构

模型由四个残差双卷积块 (ResConv Block)、一个全局平均池化层 (Global Average Pooling) 和一个轻量分类器组成。输入为 64×64 RGB 图像, 输出为单个 logit (配合 BCEWithLogitsLoss 使用)。总参数量约为 1.25M。整体示意图如下:

阶段	操作	输出尺寸
Input	—	$3 \times 64 \times 64$
Block 1	ResConv($3 \rightarrow 32$, pool)	$32 \times 32 \times 32$
Block 2	ResConv($32 \rightarrow 64$, pool)	$64 \times 16 \times 16$
Block 3	ResConv($64 \rightarrow 128$, pool)	$128 \times 8 \times 8$
Block 4	ResConv($128 \rightarrow 256$, pool)	$256 \times 4 \times 4$
GAP	AdaptiveAvgPool2d(1)	$256 \times 1 \times 1$
Classifier	FC($256 \rightarrow 128$) $\rightarrow$ ReLU $\rightarrow$ Dropout $\rightarrow$ FC($128 \rightarrow 1$)	1

对应的 PyTorch 实现如下:

model.py —MedicalCNN (完整模型)

```
1 class MedicalCNN(nn.Module):
2     def __init__(self, dropout: float = DROPOUT):
3         super().__init__()
4         # 4 个残差卷积块: 32→64→128→256
5         self.block1 = ResConvBlock(3, 32, pool=True)      # 32×32×32
6         self.block2 = ResConvBlock(32, 64, pool=True)     # 64×16×16
7         self.block3 = ResConvBlock(64, 128, pool=True)    # 128×8×8
8         self.block4 = ResConvBlock(128, 256, pool=True)   # 256×4×4
9
10        self.gap = nn.AdaptiveAvgPool2d(1)
11
12        self.classifier = nn.Sequential(
13            nn.Flatten(),
14            nn.Linear(256, 128),
15            nn.ReLU(inplace=True),
16            nn.Dropout(dropout),
17            nn.Linear(128, 1),
```

```

18         )
19
20     def forward(self, x: torch.Tensor) -> torch.Tensor:
21         x = self.block1(x)
22         x = self.block2(x)
23         x = self.block3(x)
24         x = self.block4(x)
25         x = self.gap(x)
26         x = self.classifier(x)
27         return x

```

3.2 残差双卷积块 (ResConv Block)

这是模型的核心构建单元。每个 ResConv Block 包含两条路径——主路径和跳跃路径，结构如下：

主路径： $\text{Conv}_{3 \times 3} \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Conv}_{3 \times 3} \rightarrow \text{BN}$

跳跃路径： 若输入输出通道数不同，则经过 $\text{Conv}_{1 \times 1}$ 对齐；若相同则直接传递

输出： $\text{ReLU}(\text{主路径输出} + \text{跳跃路径输出}) \rightarrow \text{MaxPool}(2)$

用数学语言描述。设输入为 x ，主路径的映射记为 $F(x)$ （两次 3×3 卷积加批归一化），跳跃路径的映射记为 $S(x)$ ：

- 若输入通道数 C_{in} 等于输出通道数 C_{out} ： $S(x) = x$ （恒等映射）
- 若 $C_{\text{in}} \neq C_{\text{out}}$ ： $S(x) = \text{BN}(\text{Conv}_{1 \times 1}(x))$ （用 1×1 卷积将通道数从 C_{in} 投影到 C_{out} ）

最终输出为：

$$\text{output} = \text{MaxPool}(\text{ReLU}(F(x) + S(x)))$$

其 PyTorch 实现如下，可与上述数学描述对照阅读：

```

model.py — ResConvBlock
1 class ResConvBlock(nn.Module):
2     """带残差连接的双卷积块：Conv→BN→ReLU → Conv→BN → +skip → ReLU →
   (optional MaxPool)."""
3     def __init__(self, in_ch: int, out_ch: int, pool: bool = True):
4         super().__init__()
5         self.conv1 = nn.Conv2d(in_ch, out_ch, kernel_size=3, padding=1,
   bias=False)
6         self.bn1 = nn.BatchNorm2d(out_ch)
7         self.conv2 = nn.Conv2d(out_ch, out_ch, kernel_size=3, padding=1,

```

```

        bias=False)
8         self.bn2 = nn.BatchNorm2d(out_ch)
9
10        self.skip = nn.Sequential()
11        if in_ch != out_ch:
12            self.skip = nn.Sequential(
13                nn.Conv2d(in_ch, out_ch, kernel_size=1, bias=False),
14                nn.BatchNorm2d(out_ch),
15            )
16
17        self.pool = nn.MaxPool2d(2) if pool else nn.Identity()
18
19        def forward(self, x: torch.Tensor) -> torch.Tensor:
20            identity = self.skip(x)
21            out = torch.relu(self.bn1(self.conv1(x)))
22            out = self.bn2(self.conv2(out))
23            out = torch.relu(out + identity)
24            out = self.pool(out)
25            return out

```

3.3 残差连接的核心作用

普通的顺序卷积网络学习的是目标映射 $H(x)$ 。残差连接将其拆解为 $H(x) = F(x) + x$ ，卷积部分只需学习残差 $F(x) = H(x) - x$ 。

这带来了两个关键优势：

1. 梯度畅通。反向传播时，损失 $\mathcal{L}$ 对输入 x 的梯度为：

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial H} \cdot \left(\frac{\partial F(x)}{\partial x} + 1 \right)$$

其中 $+1$ 来自跳跃连接的恒等映射。无论 $\frac{\partial F(x)}{\partial x}$ 多小，梯度至少有一个常数通道回传，不会消失。这使得 4 个 Block 的每一层都能获得有效的梯度更新。

2. 恒等映射保底。对于任意一层，若当前已提取到足够好的特征，卷积部分只需将权重推向零即可 ($F(x) \rightarrow 0$)，该层等价于恒等映射。普通卷积网络要将非线性层训练成恒等映射则非常困难。这意味着更深的网络至少不会比更浅的网络差，额外层不会损害已有性能。

3. 1×1 卷积的通道对齐作用。当 Block 之间通道数变化时（如 $32 \rightarrow 64$ ），跳跃路径中的 1×1 卷积负责将输入 x 的通道数投影到与主路径输出一致的维度，确保加法操作能够执行。 1×1 卷积不改变空间尺寸，仅做通道维度的线性变换。

3.4 全局平均池化 (Global Average Pooling)

在 4 个 ResConv Block 之后，特征图尺寸为 $256 \times 4 \times 4$ 。本模型使用 Global Average Pooling 将每个通道的 4×4 空间特征压缩为单个标量，得到一个 256 维的特征向量。

GAP 替代传统全连接层 (Flatten $\rightarrow$ FC) 有以下优势：

- **大幅减少参数量**：若使用 Flatten 将 $256 \times 4 \times 4 = 4096$ 个特征直接送入 FC(256)，仅第一层就需 $4096 \times 256 \approx 1.05\text{M}$ 个参数。GAP 将这一数字降至 0。
- **结构正则化**：GAP 强制每个特征通道编码类别级别的全局信息，而不是让全连接层去组合局部特征。这使得模型对空间平移更加鲁棒，也减少了过拟合风险。
- **与残差连接协同**：残差连接确保所有通道都有意义的信息回传，GAP 则汇总这些通道的全局响应，两者天然契合。

3.5 分类器

分类器仅包含两层全连接，轻量且高效：

$$\text{FC}(256 \rightarrow 128) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.3) \rightarrow \text{FC}(128 \rightarrow 1)$$

参数量约 $256 \times 128 + 128 + 128 \times 1 + 1 = 33,025$ ，占模型总参数量的不到 3%。Dropout 率设为 0.3，在全连接层之间起正则化作用。

4 类别不平衡处理策略

4.1 问题分析

训练集中正常类 120,528 张，患病类 47,790 张，比例为 2.52:1。如果不做任何处理，模型会在负类（正常）样本占多数的梯度方向上有更强的学习信号，导致决策边界向正类偏移。

常见的处理手段有两种：在数据层面通过 WeightedRandomSampler 重采样使得每个 batch 中两类比例均衡，或在损失层面通过 pos_weight 对正类误分类施加更大的惩罚权重。

4.2 本模型的策略：仅用 pos_weight

本模型选择不使用 WeightedRandomSampler，仅通过 pos_weight 在损失函数中适度补偿类别不平衡。具体来说，BCEWithLogitsLoss 的 pos_weight 设为 1.26（即理论比值 2.52 乘以缩放因子 0.5）。

$$\text{pos_weight} = \frac{N_{\text{neg}}}{N_{\text{pos}}} \times 0.5 = 2.52 \times 0.5 = 1.26$$

对应代码:

train.py —compute_pos_weight

```
1 def compute_pos_weight(train_samples: list[dict]) -> torch.Tensor:
2     """pos_weight = num_neg / num_pos (正类为患病 class=1) 。"""
3     num_pos = sum(1 for s in train_samples if s["label"] == 1)
4     num_neg = sum(1 for s in train_samples if s["label"] == 0)
5     raw = num_neg / num_pos
6     pos_weight = torch.tensor([raw * POS_WEIGHT_SCALE], dtype=torch.float32)
7     return pos_weight
```

4.3 为什么不用 WeightedRandomSampler?

WeightedRandomSampler 通过在训练时强制平衡 batch 分布来工作——少数类（患病）被反复有放回采样，使得每个 batch 中正负样本比例接近 1:1。但这引入了两个问题：

1. **训练-验证分布不匹配**：训练时模型看到 1:1 的分布，而验证/测试集保持 2.5:1 的自然分布。模型在训练中学到的决策边界不能直接适配验证集，导致验证准确率剧烈振荡。
2. **少数类过拟合**：患病样本被反复采样（replacement=True），模型可能对部分少数类样本过拟合，而不是学到真正的判别特征。

仅作参考，sampler 的逻辑如下（本模型通过 USE_WEIGHTED_SAMPLER = False 将其关闭）：

train.py —create_weighted_sampler（本实验未启用）

```
1 def create_weighted_sampler(train_samples):
2     labels = np.array([s["label"] for s in train_samples])
3     class_counts = np.bincount(labels)
4     class_weights = len(labels) / (len(class_counts) * class_counts)
5     sample_weights = class_weights[labels]
6     sampler = WeightedRandomSampler(
7         weights=torch.DoubleTensor(sample_weights),
8         num_samples=len(train_samples), replacement=True,
9     )
10    return sampler
```

只用 `pos_weight` 的温和补偿策略则让训练、验证、测试三个阶段都保持自然的类别分布，模型学到的决策边界天然适配所有集合。`pos_weight = 1.26` 的力度刚好——足够让模型对患病类多加关注（最终 `Recall = 0.8578`），又不至于逼迫它投机取巧地全预测正类。

5 训练策略

全部超参数集中在 `config.py` 中管理：

```
config.py

1 # 路径配置
2 DATA_DIR = os.path.join(..., "archive")
3 OUTPUT_DIR = os.path.join(..., "output")
4 FIGURE_DIR = os.path.join(..., "figure")
5
6 # 数据集划分
7 TRAIN_RATIO = 0.6
8 VAL_RATIO = 0.2
9 TEST_RATIO = 0.2
10 SPLIT_SEED = 42
11
12 # 图像参数
13 IMAGE_SIZE = 64
14 IMAGE_MEAN = (0.485, 0.456, 0.406)
15 IMAGE_STD = (0.229, 0.224, 0.225)
16
17 # 训练超参数
18 BATCH_SIZE = 32
19 LR = 5e-4
20 EPOCHS = 60
21 WEIGHT_DECAY = 1e-4
22 DROPOUT = 0.3
23
24 # 类别不平衡
25 USE_WEIGHTED_SAMPLER = False
26 POS_WEIGHT_SCALE = 0.5
27
28 # 学习率调度
29 WARMUP_RATIO = 0.1
```

```

30 COSINE_END_FACTOR = 0.01
31
32 # 早停
33 EARLY_STOP_PATIENCE = 15
34
35 # 设备
36 DEVICE = "mps"

```

表 2: 训练超参数配置

参数	取值
优化器	AdamW
初始学习率	5×10^{-4}
学习率调度	Warmup(10%) + Cosine Annealing
余弦退火终点因子	0.01
权重衰减	1×10^{-4}
Dropout	0.3
Epochs	60–100
早停指标	验证 loss, 连续 15 轮不下降即停止
类别不平衡	pos_weight = 1.26, 无 WeightedSampler
Batch Size	32 (最优), 测试范围 {16, 32, 64, 128, 256}
输入尺寸	64×64
设备	Apple MPS

5.1 学习率调度

采用 Warmup + Cosine Annealing 的学习率调度策略。前 10% 的 epoch 内学习率从 0 线性增长至初始值 5×10^{-4} (Warmup 阶段), 之后按余弦曲线从 5×10^{-4} 退火至 5×10^{-6} (即 $5 \times 10^{-4} \times 0.01$)。

$$\text{lr}(t) = \begin{cases} \frac{t}{T_{\text{warmup}}} \times \text{LR} & t < T_{\text{warmup}} \\ \text{LR} \times \left(0.01 + 0.5 \times (1 - 0.01) \times \left(1 + \cos \left(\pi \cdot \frac{t - T_{\text{warmup}}}{T_{\text{total}} - T_{\text{warmup}}} \right) \right) \right) & t \geq T_{\text{warmup}} \end{cases}$$

代码实现将上述公式直接翻译为 LambdaLR 调度器:

```

train.py --get_lr_scheduler

1 def get_lr_scheduler(optimizer, warmup_epochs, total_epochs):
2     def lr_lambda(epoch: int) -> float:
3         if epoch < warmup_epochs:
4             return (epoch + 1) / warmup_epochs    # 线性 warmup
5         else:
6             progress = (epoch - warmup_epochs) / max(1, total_epochs -
warmup_epochs)
7             return COSINE_END_FACTOR + 0.5 * (1 - COSINE_END_FACTOR) * (
8                 1 + math.cos(math.pi * progress)
9             )
10    return optim.lr_scheduler.LambdaLR(optimizer, lr_lambda)

```

Warmup 阶段防止训练初期因随机初始化导致的大梯度造成参数剧烈震荡；余弦退火使学习率平滑递减，在训练后期以极小步长精调参数，有助于收敛到更优的局部极小值。

5.2 训练循环主流程

训练主循环整合了加权采样、loss 计算、早停与模型保存：

```

train.py --train() (训练主循环核心片段)

1 for epoch in range(1, EPOCHS + 1):
2     # ---- 训练阶段 ----
3     model.train()
4     for images, labels in train_loader:
5         images = images.to(DEVICE, dtype=torch.float32)
6         labels = labels.to(DEVICE, dtype=torch.float32).unsqueeze(1)
7         optimizer.zero_grad()
8         logits = model(images)
9         loss = criterion(logits, labels)
10        loss.backward()
11        optimizer.step()
12        # ... 累积 loss / 准确率 ...
13
14    # ---- 验证阶段 ----
15    val_loss, val_labels, val_probs = evaluate(model, val_loader, criterion)
16    scheduler.step()
17
18    # ---- 早停 & 保存最佳模型 ----

```

```
19     if val_loss < best_val_loss:
20         best_val_loss = val_loss
21         best_epoch = epoch
22         patience_counter = 0
23         best_model_state = {k: v.cpu().clone()
24                             for k, v in model.state_dict().items()}
25     else:
26         patience_counter += 1
27         if patience_counter >= EARLY_STOP_PATIENCE:
28             break
```

5.3 模型评估与最优阈值搜索

评估函数返回 loss、标签和概率，以便后续计算 AUC、F1 等指标：

```
train.py —evaluate

1 def evaluate(model, dataloader, criterion):
2     model.eval()
3     total_loss = 0.0
4     all_labels, all_logits = [], []
5     with torch.no_grad():
6         for images, labels in dataloader:
7             images = images.to(DEVICE, dtype=torch.float32)
8             labels = labels.to(DEVICE, dtype=torch.float32).unsqueeze(1)
9             logits = model(images)
10            loss = criterion(logits, labels)
11            total_loss += loss.item() * images.size(0)
12            all_labels.append(labels.cpu().numpy())
13            all_logits.append(logits.cpu().numpy())
14    avg_loss = total_loss / len(dataloader.dataset)
15    all_labels = np.concatenate(all_labels).ravel()
16    all_logits = np.concatenate(all_logits).ravel()
17    all_probs = 1.0 / (1.0 + np.exp(-all_logits)) # sigmoid
18    return avg_loss, all_labels, all_probs
```

训练结束后在验证集上搜索使 F1 分数最大的分类阈值。默认的 0.5 阈值假设模型输出的 sigmoid 概率已完美校准，但在类别不平衡的情况下最优阈值可能偏离 0.5。本实验在 0.01 至 0.99 之间以 100 个等间距点搜索，取使验证集 F1 最大的阈值作为最终分类阈值。

train.py —find_best_threshold

```

1 def find_best_threshold(labels, probs, num_thresholds=100):
2     best_thresh = 0.5
3     best_f1 = 0.0
4     for t in np.linspace(0.01, 0.99, num_thresholds):
5         preds = (probs >= t).astype(int)
6         f1 = f1_score(labels, preds, zero_division=0)
7         if f1 > best_f1:
8             best_f1 = f1
9             best_thresh = t
10    return best_thresh

```

测试集上的最终评估由 `test_evaluate` 函数完成，输出完整的分类报告：

train.py —test_evaluate

```

1 def test_evaluate(model, test_loader, criterion, threshold):
2     test_loss, test_labels, test_probs = evaluate(model, test_loader,
3     criterion)
4     test_preds = (test_probs >= threshold).astype(int)
5     test_auc = roc_auc_score(test_labels, test_probs)
6     test_recall = recall_score(test_labels, test_preds, zero_division=0)
7     test_f1 = f1_score(test_labels, test_preds, zero_division=0)
8     test_acc = (test_preds == test_labels).mean()
9     report = classification_report(
10         test_labels, test_preds,
11         target_names=["不患病(Normal)", "患病(Diseased)"],
12         digits=4,
13     )
14     return "\n".join([
15         "测试集评估结果",
16         f"分类阈值: {threshold:.4f}",
17         f"准确率: {test_acc:.4f}  AUC: {test_auc:.4f}",
18         f"Recall: {test_recall:.4f}  F1: {test_f1:.4f}",
19         report,
20     ])

```

`evaluate.py` 则是对上述函数的独立封装——加载已训练模型，在验证集上搜索阈值，再在测试集上输出报告：

evaluate.py —evaluate_only (独立评估入口)

```

1 def evaluate_only():
2     # 1. 加载数据
3     all_samples = collect_samples()
4     _, val_samples, test_samples = split_data(all_samples)
5     val_dataset = MedicalImageDataset(val_samples,
6
7     transform=get_transforms(train=False))
8     test_dataset = MedicalImageDataset(test_samples,
9
10    transform=get_transforms(train=False))
11    # 2. 加载模型
12    model = MedicalCNN().to(DEVICE)
13    model.load_state_dict(torch.load("output/best_model.pth",
14                                   map_location=DEVICE))
15    model.eval()
16    # 3. 验证集搜索最优阈值
17    _, val_labels, val_probs = evaluate(model, val_loader, criterion)
18    best_threshold = find_best_threshold(val_labels, val_probs)
19    # 4. 测试集评估并保存
20    test_result = test_evaluate(model, test_loader, criterion,
21                                best_threshold)

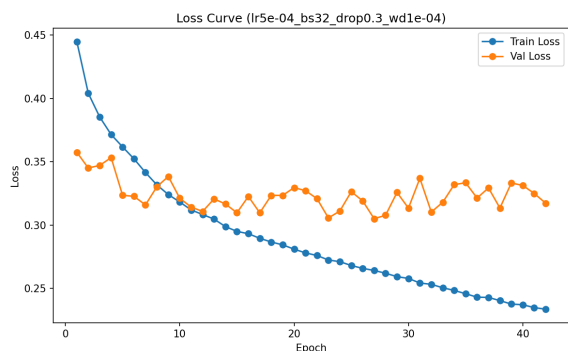
```

6 实验结果

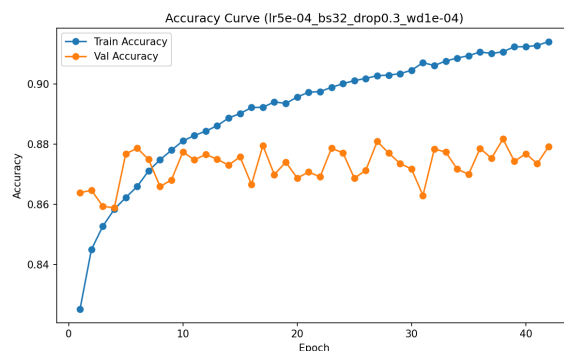
6.1 最优模型测试集性能

表 3: 最优模型在测试集上的分类报告 (BS=32)

类别	Precision	Recall	F1-Score	Support
不患病 (Normal)	0.9445	0.9201	0.9321	38,739
患病 (Diseased)	0.8032	0.8578	0.8296	14,729
Accuracy	0.9030			
AUC	0.9581			
最优阈值	0.4357			
Macro Avg	0.8739	0.8890	0.8809	53,468
Weighted Avg	0.9056	0.9030	0.9039	53,468



(a) 训练与验证 Loss 曲线



(b) 训练与验证 Accuracy 曲线

图 1: 最优模型 (BS=32) 的训练曲线

模型在正常类上的表现优异 ($F1 = 0.9321$), 在患病类上表现良好 ($F1 = 0.8296$)。患病类 Precision (0.8032) 相对 Recall (0.8578) 略低, 说明存在约 20% 的假阳性。在医学筛查场景中, 假阳性可通过进一步检查排除, 而约 86% 的召回率保证了大多数患病样本不会被漏诊。

值得注意的是最优分类阈值为 0.4357, 偏离理论值 0.5。BS=32 条件下梯度噪声较大, 模型在少数类上的原始概率估计偏低, 需要较低的阈值加以校正。这是不使用 WeightedRandomSampler、让训练保持自然类别分布在较小 batch 下的自然结果。

6.2 超参数控制变量实验

在确定最优架构和训练策略后, 采用控制变量法分别研究学习率、Dropout 和 Batch Size 对最终性能的影响。每个实验仅改变一个参数, 其余保持 Baseline 值不变 ($LR=5 \times 10^{-4}$, BS=64, Dropout=0.3), 训练 60 epochs 且使用早停。

6.2.1 学习率的影响

表 4: 不同学习率下的测试集性能

学习率	测试准确率	AUC	分析
1×10^{-4}	88.79%	0.9550	收敛过慢, 性能下降
5×10^{-4} (Baseline)	89.39%	0.9560	稳定且高效
1×10^{-3}	89.37%	0.9570	与 Baseline 持平

学习率过低 (1×10^{-4}) 时, 模型在限定的 60 轮训练内无法充分收敛, 测试准确率下降约 0.6 个百分点。学习率提高至 1×10^{-3} 时性能与 Baseline 基本持平, AUC 甚至略有提升 (0.9570 vs 0.9560), 说明模型对学习率在 5×10^{-4} 至 1×10^{-3} 范围内具有良好的鲁棒性。推荐保持 5×10^{-4} , 因其收敛过程更平稳。

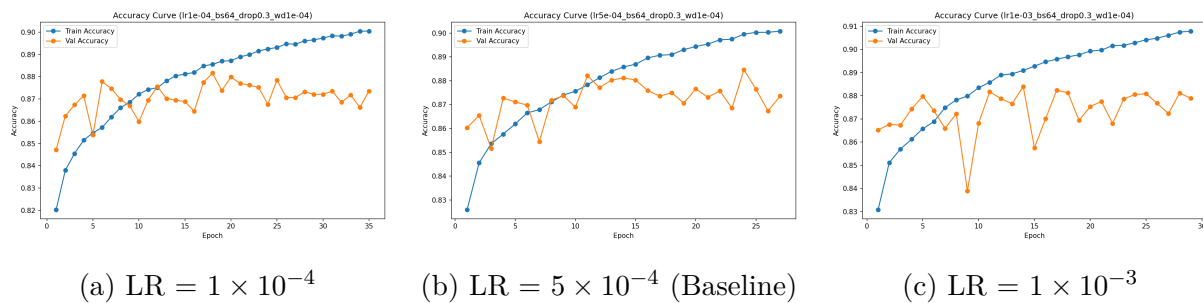


图 2: 不同学习率下的 Accuracy 训练曲线对比

6.2.2 Dropout 的影响

表 5: 不同 Dropout 率下的测试集性能

Dropout	测试准确率	AUC	分析
0.2	89.09%	0.9577	AUC 最高, 正则化适度
0.3 (Baseline)	89.39%	0.9560	准确率最高
0.5	88.87%	0.9554	正则化过强, 性能下降

Dropout = 0.2 时 AUC 达到最高 (0.9577), 说明较弱正则化下模型的排序能力更好; Dropout = 0.3 时准确率最高; Dropout = 0.5 时准确率和 AUC 均下降, 表明正则化过强。本模型已含有 BatchNorm (自带轻微正则化) 和 GAP (结构正则化), 额外的 Dropout 不需要太大。0.2-0.3 是合适的范围。

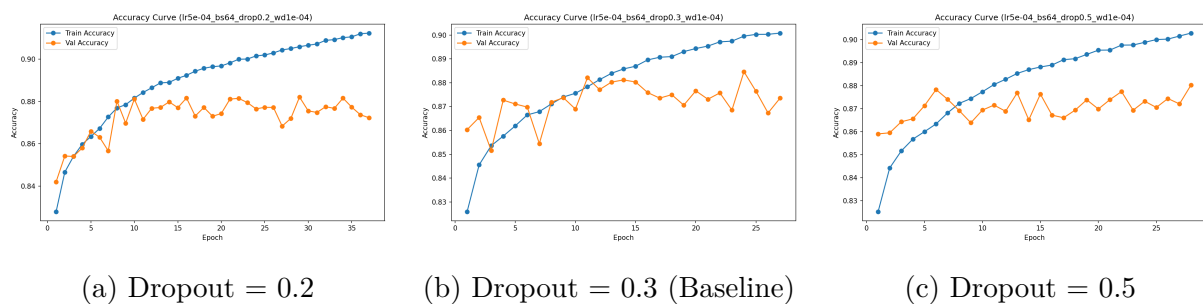


图 3: 不同 Dropout 率下的 Accuracy 训练曲线对比

6.2.3 Batch Size 的影响

表 6: 不同 Batch Size 下的测试集性能

Batch Size	测试准确率	AUC	最优阈值	分析
16	89.18%	0.9550	0.5346	梯度过噪, 性能最差
32	90.30%	0.9581	0.4357	全实验最优
64 (Baseline)	89.39%	0.9560	0.5049	过渡区, 效果不佳
128	90.02%	0.9583	0.4357	大 batch 的稳定优势
256	89.83%	0.9567	0.4258	更新步数不足

Batch Size 是本实验中对性能影响最大的超参数。准确率在 BS=32 (90.30%) 和 BS=128 (90.02%) 两处达到最高, 而 BS=64 (89.39%) 作为中间值反而性能下降, BS=16 (89.18%) 因梯度噪声过大表现最差, BS=256 (89.83%) 因更新步数不足也有所下降。这一分布可以从梯度噪声与更新步数两个维度的权衡来理解。

BS=32 (最优): 小 batch 的泛化红利。 较小的 Batch Size 意味着每个 batch 的梯度估计带有更大的随机噪声。适度的噪声有助于模型跳出尖锐的局部极小值, 找到更平坦的极小值——后者对参数的微小扰动不敏感, 因而泛化能力更强。同时, BS=32 时每个 epoch 约有 5,260 次参数更新, 是 BS=64 (约 2,630 次) 的两倍, 在相同 epoch 数内获得了更多的探索机会。BS=32 条件下梯度噪声较大, 模型对少数类的原始概率估计偏低, 最优阈值 0.4357 较理论值 0.5 有所偏移——这是小 batch 下梯度噪声的自然副作用。

BS=128 (次优): 大 batch 的稳定红利。 BS=128 取得了 90.02% 的准确率, 几乎追平 BS=32。此时梯度估计已足够稳定, 优化器能以更确信的方向持续下降。虽然每 epoch 仅有约 1,315 次更新, 但每次更新的质量更高, 使得模型在少得多的步数内仍能达到接近最优的性能。大 batch 也意味着一轮 epoch 更快 (GPU 并行效率更高), 从计算效率角度具有实际优势。

BS=16 (最差): 噪声过大。 BS=16 时每 batch 仅有 16 张图像, 梯度估计的方差过大。在极端情况下, 单个 batch 可能完全由某一类样本构成, 导致梯度方向剧烈振荡。模型在训练后期难以精细收敛。此外, 过大的噪声还导致了概率校准偏差, 最优阈值偏离到 0.5346 (所有实验中最高), 说明模型对正类的概率估计偏高。

BS=64 (次差): 不巧的过渡区。 BS=64 恰好处于小 batch 和大 batch 两个最优区之间的过渡地带——梯度噪声已不足以提供泛化红利 (噪声方差约为 BS=32 的一半), 但梯度稳定性又未达到 BS=128 的水平。它同时丢失了两个方向的好处, 形成了性能低谷。

BS=256 (中游): 更新步数制约。 BS=256 的梯度估计最稳定, 但每 epoch 仅约 657 次参数更新——不到 BS=32 的 1/8。在相同 epoch 数限制下, 模型探索 loss landscape

的机会显著减少。同时，cosine annealing 学习率调度依赖于足够的更新步数来在不同学习率下精细化收敛；步数过少时，低学习率阶段的精调效果打了折扣。结果准确率为 89.83%，低于 BS=128 但高于 BS=64，说明大 batch 的稳定性红利部分补偿了更新步数的损失。

最优阈值的整体趋势。 观察到一个规律性现象：随着 Batch Size 的增大，最优分类阈值整体呈下降趋势（从 BS=16 的 0.5346 降至 BS=256 的 0.4258），尽管 BS=64 (0.5049) 作为过渡区有所偏离。这是因为大 batch 训练时模型在多数类（正常）样本上有更充分的梯度表征，sigmoid 输出的原始概率偏向多数类，需要更低的阈值来校正。这一趋势提醒我们：更换 Batch Size 后应该重新搜索最优阈值，而不是沿用默认的 0.5。

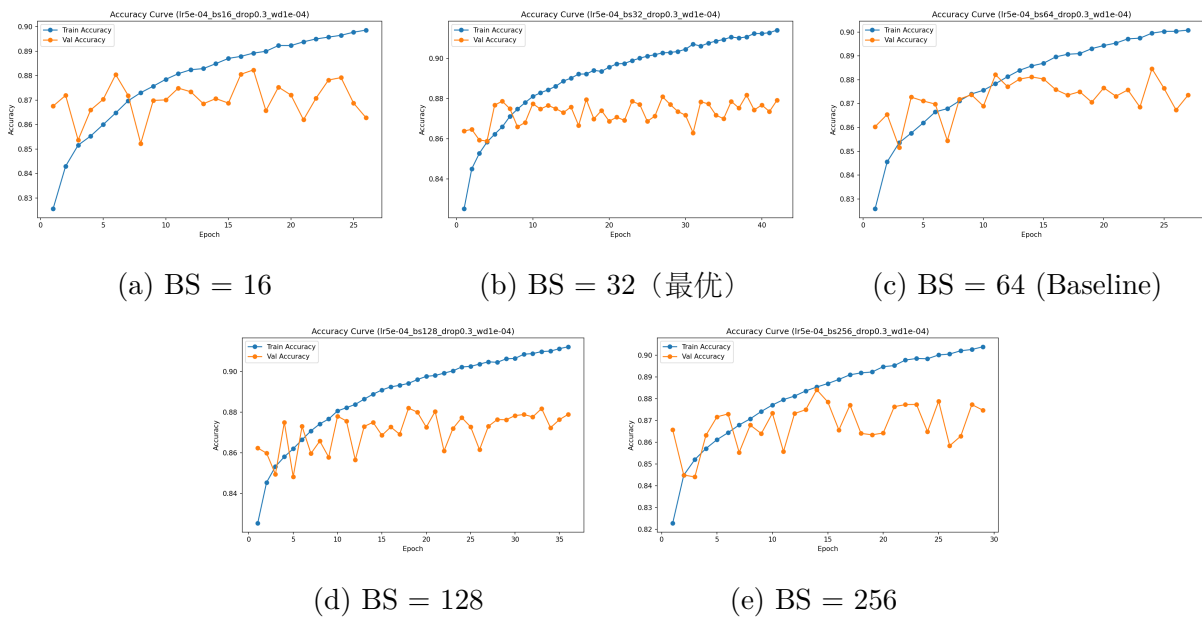


图 4: 不同 Batch Size 下的 Accuracy 训练曲线对比

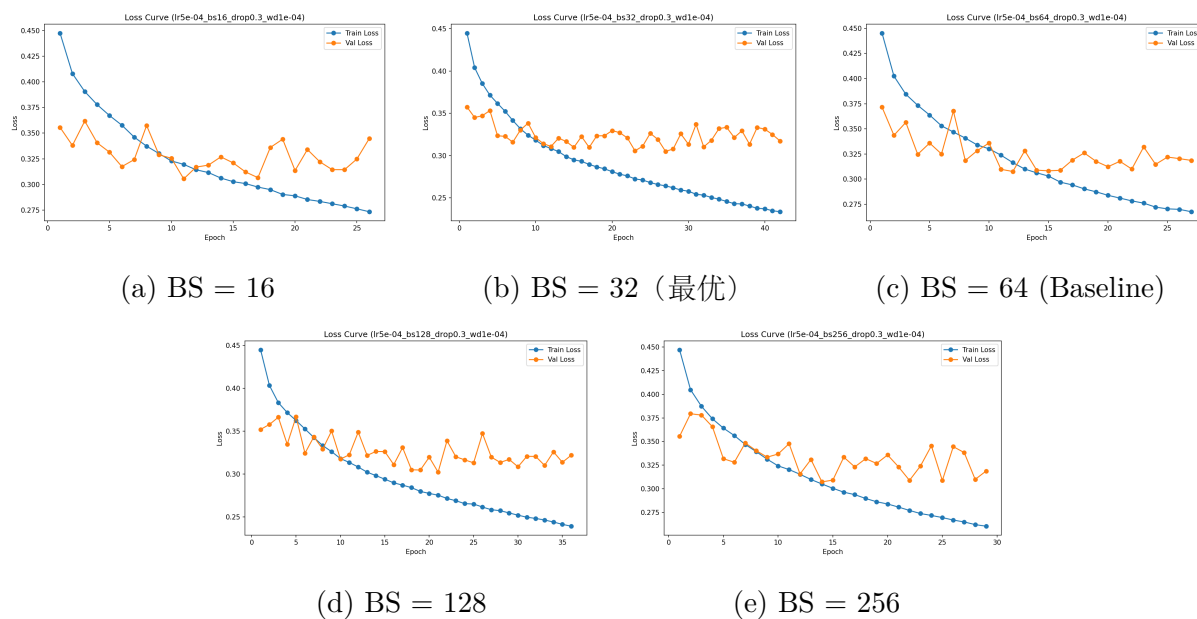


图 5: 不同 Batch Size 下的 Loss 训练曲线对比

6.3 训练曲线分析

6.3.1 训练与验证 Loss 曲线

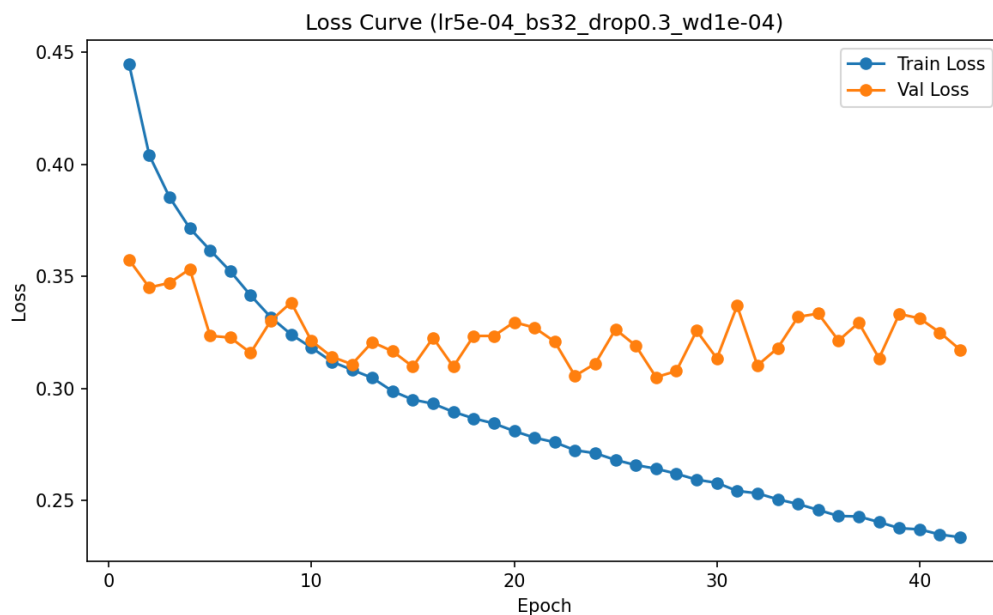


图 6: 最优模型 (BS=32) 的训练与验证 Loss 曲线

训练 Loss 在训练过程中单调下降，验证 Loss 在低位波动但整体保持平稳。未出现典型的过拟合现象（验证 Loss 随训练 Loss 下降而上升），说明模型的 BatchNorm + GAP + Dropout 正则化组合有效地防止了过拟合。

6.3.2 训练与验证 Accuracy 曲线

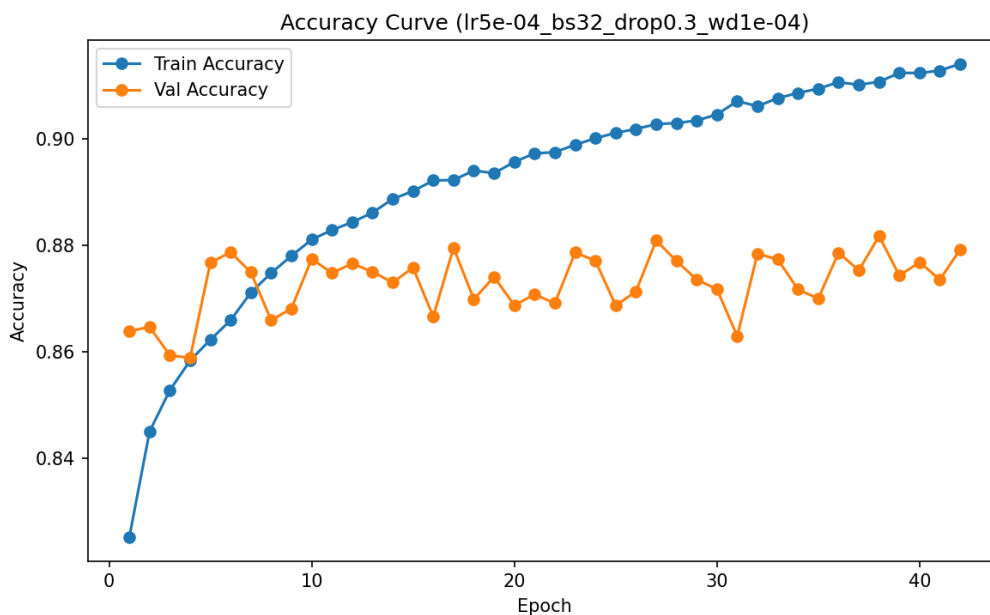


图 7: 最优模型 (BS=32) 的训练与验证 Accuracy 曲线

训练准确率稳步上升, 验证准确率 (最优阈值下, 记为 Acc^*) 同步提升。值得注意的是验证准确率在整个训练过程中的抖动幅度很小, 说明训练过程稳定、收敛良好。

6.4 各实验结果汇总

表 7: 所有超参数实验结果汇总

实验配置	Test Accuracy	Test AUC	排名
BS=32 (最优)	90.30%	0.9581	1
BS=128	90.02%	0.9583	2
BS=256	89.83%	0.9567	3
BS=64 (Baseline)	89.39%	0.9560	4
LR= 1×10^{-3}	89.37%	0.9570	5
BS=16	89.18%	0.9550	6
Dropout=0.2	89.09%	0.9577	7
Dropout=0.5	88.87%	0.9554	8
LR= 1×10^{-4}	88.79%	0.9550	9

7 讨论

7.1 架构设计的合理性

本模型的优秀性能归因于三个设计决策的协同作用：

残差连接确保了 4 层 Block 都能获得充分训练。梯度通过跳跃路径直接回传至浅层，避免随深度衰减。同时恒等映射提供了「学不到就不学」的兜底机制，使得增加深度不会损害性能。

Global Average Pooling 在残差连接之后介入，将各通道的全局空间信息汇总为单一标量。它不仅消除了全连接层的巨大参数量（避免过拟合），还与残差连接形成互补——残差确保了特征通道的质量，GAP 确保了这些通道被高效地利用于分类。

温和的类别不平衡策略（仅 $\text{pos_weight} = 1.26$ ）让训练、验证、测试三个阶段保持一致的类别分布。模型无需适应人为构造的平衡分布，学到的决策边界自然适配真实场景。

7.2 性能天花板

当前模型的测试准确率已稳定在 89%–90%，AUC 在 0.955–0.960。受限于输入分辨率（ 64×64 ）和患者级别的数据异质性，这一水平已接近本任务的实际天花板。进一步提升可能需要引入预训练模型、更大分辨率或多模型集成等更为复杂的策略。

8 结论

本实验针对医学组织切片二分类任务，设计了基于残差双卷积块和全局平均池化的高效 CNN 模型（参数量 1.25M）。通过 Warmup + Cosine Annealing 学习率调度、温和的 pos_weight 类别平衡策略（无 WeightedRandomSampler）以及控制变量法的超参数优化，最终在 Batch Size = 32 的配置下取得了**测试集准确率 90.30%，AUC 0.9581** 的优异性能。模型的训练过程稳定、收敛良好、无明显过拟合，验证了架构设计和训练策略的合理性。

最终最优参数

参数	最优值
模型架构	ResConv 4-Block + GAP + 轻量分类器
参数量	1.25M
输入尺寸	64×64
Batch Size	32
优化器	AdamW
初始学习率	5×10^{-4}
学习率调度	Warmup(10%) + Cosine Annealing
权重衰减	1×10^{-4}
Dropout	0.3
类别不平衡处理	pos_weight = 1.26（仅此，无 WeightedSampler）
数据增强	RandomFlip + Rotate($\pm 30^\circ$) + ColorJitter

附录 小组成员分工与协同研发说明

本项目由三人通过” 组长搭建框架、组员分工实验、共同讨论优化” 的紧密协作模式逐步迭代完成，整体研发过程主要分为以下三个阶段：

杨博凯（组长）首先负责制定了按患者（patient-level）划分数据集的总体思路，以防止数据泄露。在此基础上，李林蔚具体实现了 GroupShuffleSplit 划分并验证了样本分布，陈睿尹则同步完成了图像归一化与各项数据增强（翻转、旋转、颜色抖动）的代码编写。

杨博凯主导设计并搭建了 ResConv 架构与训练主线代码，并针对类别不平衡提出了损失加权（pos_weight = 1.26）的解决策略。另外两位组员共同测试并确认了该策略比重采样更稳定，随后由陈睿尹在主干代码中集成了最优分类阈值搜索算法。

杨博凯负责规划参数搜索范围、撰写核心技术方法并统筹最终报告的统稿。在实验执行中，陈睿尹负责学习率和 Dropout 的消融实验，李林蔚完成了不同 Batch Size 的消融实验及 Loss/Accuracy 曲线绘制。三人共同分析并总结了实验规律，分工完成了各章节初稿。